

BASICS OF FIM · WI001287

Statistical Learning

2: Statistical Learning & bias-variance trade-off

Univ.-Prof. Dr.-Ing. habil. Niklas Kühl
Lars Böcking

Chair for Information Systems and
Human-centric Artificial Intelligence
University of Bayreuth
TUM School of Management

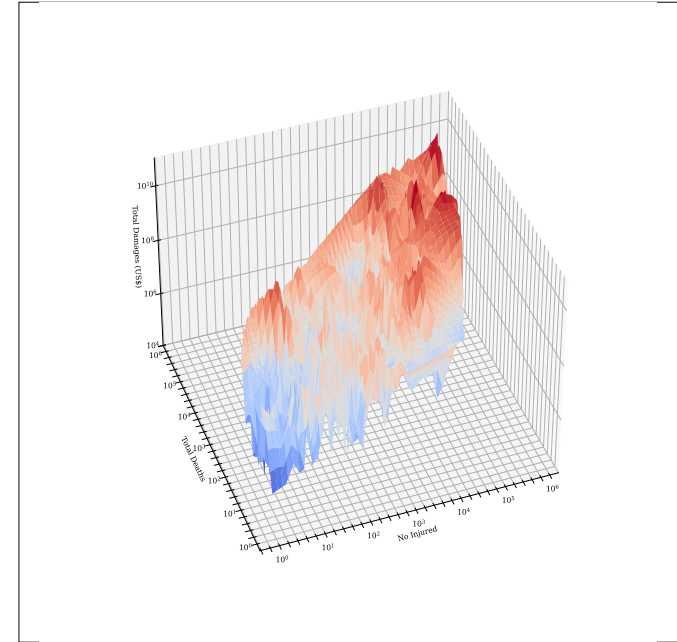
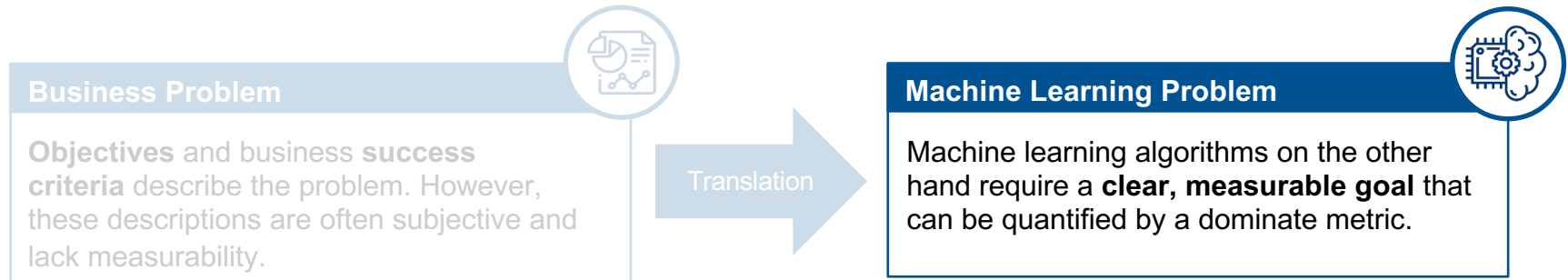


Figure 2.2: Function surface of total damages (US\$)

- Translating business problems to machine learning problems
- Reducible and irreducible error terms in statistical learning
- Bias variance trade off
- Classification & Regression
- Data splitting

- Translating business problems to machine learning problems
- Reducible and irreducible error terms in statistical learning
- Bias variance trade off
- Classification & Regression
- Data splitting

Problem Statement: Machine learning algorithms need measurable goals.



We look at real world data from NASA throughout FIM-Basics.

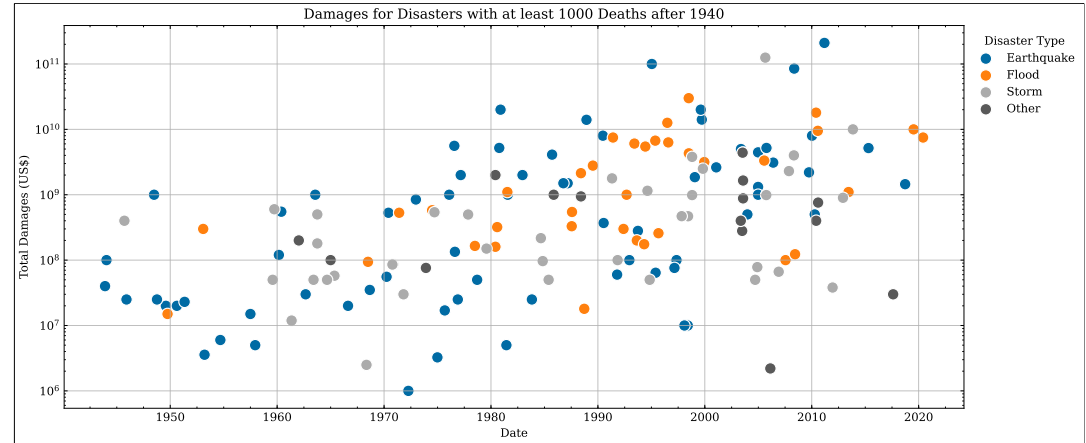


Figure 1.1: damage distribution

- Real world data from NASA on natural disasters around the world from 1900-2021 covering >16K events [3]
- Disasters such as Storm, Drought, Earthquake, Epidemic, Flood, Volcanic activity among others
- Impact reported by number of death, number of homeless, total damage, etc.

[1] <https://www.earthdata.nasa.gov>

[2] <https://www.nature.com/articles/s41597-021-00846-6>

[3] <https://www.kaggle.com/datasets/brsincier/all-natural-disasters-19002021-eosdis>

Differentiate Features and target variables in a dataset.

Feature: an individual measurable property or characteristic of a phenomenon

Target: is the variable the model is trying to predict

ID	Feature 1	Feature 2	...	Feature p	Target
1					
2					
3					
4					
5					
...					
n					

Model **learns** relationships from input to a target variable on historical data....

...and then **applies** this knowledge to instances with unknown target variables.

 EARTHDATA

Date	Total Deaths ('000)	Home-less ('000)	Disaster type	Total Damage (Mrd. US \$)
1963-09-07	12.5	24.2	Flood	13.2
1979-05-21	7.8	22.5	Earthquake	2.5
1981-03-11	18.2	26.8	Storm	26.7
2001-11-09	9.3	23.7	Earthquake	3.3
2005-01-05	0.7	25.1	Volcanic activity	6.9
2008-06-27	23.0	28.5	Earthquake	32.9
2021-08-13	6.1	21.4	Flood	9.5

Learn relation between features and target variables in a dataset.

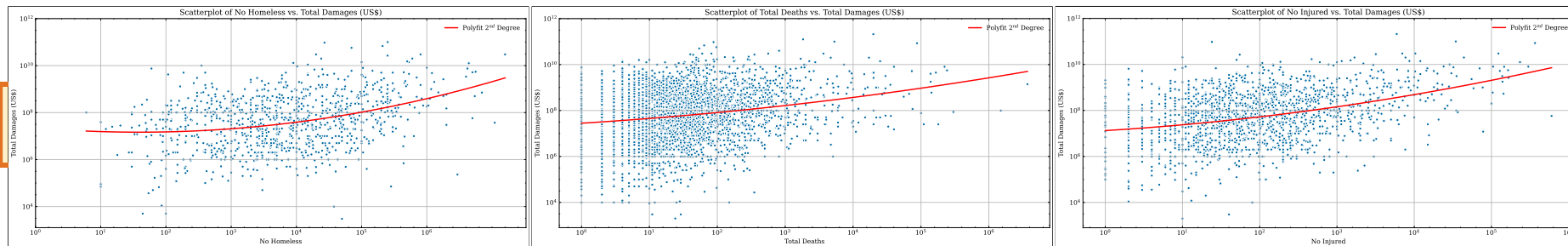


Figure 2.1: feature target relation

Shown are *Total Damage (US\$)* vs *Homeless, Total Death and Injured*, with a red regression fit separately to each.

Can we predict *Total Damage (US\$)* using these three?

Perhaps we can do better using a **model**:

$$\textit{Total Damage (US\$)} \approx f(\textit{Homeless, Total Death, Injured})$$

Agenda.

- Translating business problems to machine learning problems
- Reducible and irreducible error terms in statistical learning
- Bias variance trade off
- Classification & Regression
- Data splitting
- Hypothesis testing

Learn a function to map from X to Y .

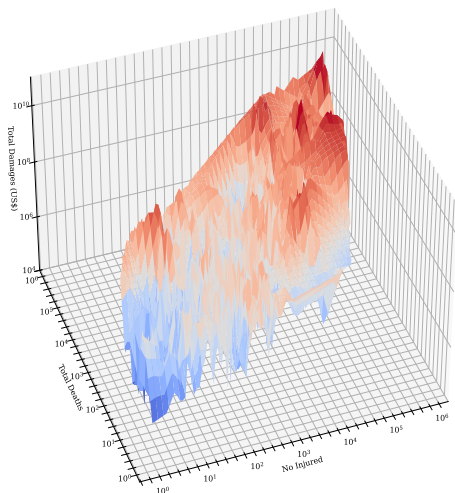


Figure 2.2: function surface

$$\text{Total Damage (US\$)} \approx f(\text{Homeless, Total Death, Injured})$$

$$Y \approx f(X_1, X_2, X_3) + \varepsilon$$

- Where ε captures measurement errors and other discrepancies.

Or without specifying the number of features, we believe about the world that: $Y \approx f(X) + \varepsilon$

And we want to learn this function:

$$\hat{Y} \approx \hat{f}(X)$$

Function as a mapping from X to Y , not to be confused with the programming concept in python.

- Where $\hat{f}$ represents our estimate for f and $\hat{Y}$ our resulting estimation for a given X .

Why have a function from X to Y in the first place?

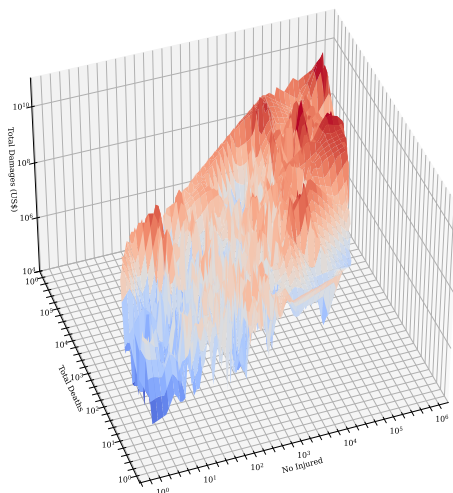


Figure 2.2: function surface

$$\text{Total Damage (US\$)} \approx f(\text{Homeless, Total Death, Injured})$$

$$\hat{Y} \approx \hat{f}(X)$$

- With a good $\hat{f}$ we can make predictions of Y at new points $X = x$
- We can understand which components of $X = (X_1, X_2, \dots, X_p)$ are important in explaining Y

When do we have a good function from X to Y ?

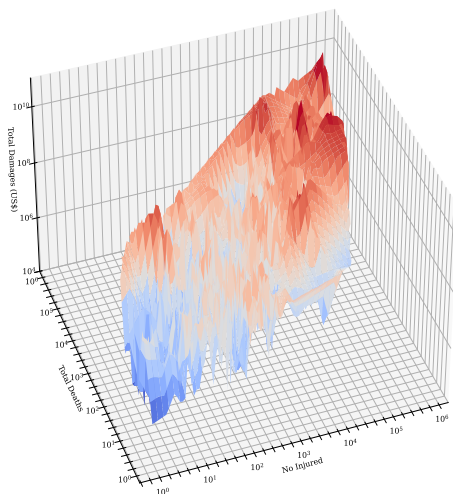


Figure 2.2: function surface

$$\text{Total Damage (US\$)} \approx f(\text{Homeless, Total Death, Injured})$$

$$Y \approx f(X) + \varepsilon$$

Where ε captures measurement errors and other discrepancies.

So even if we knew $f(X)$, which we don't, we still observe an error for an ideal predictor:

$$\varepsilon = Y - f(X)$$

→ We call this the **irreducible error**

When do we have a good function from X to Y ?

 EARTHDATA

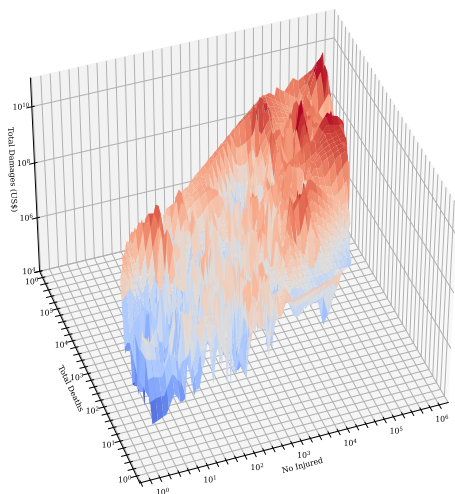


Figure 2.2: function surface

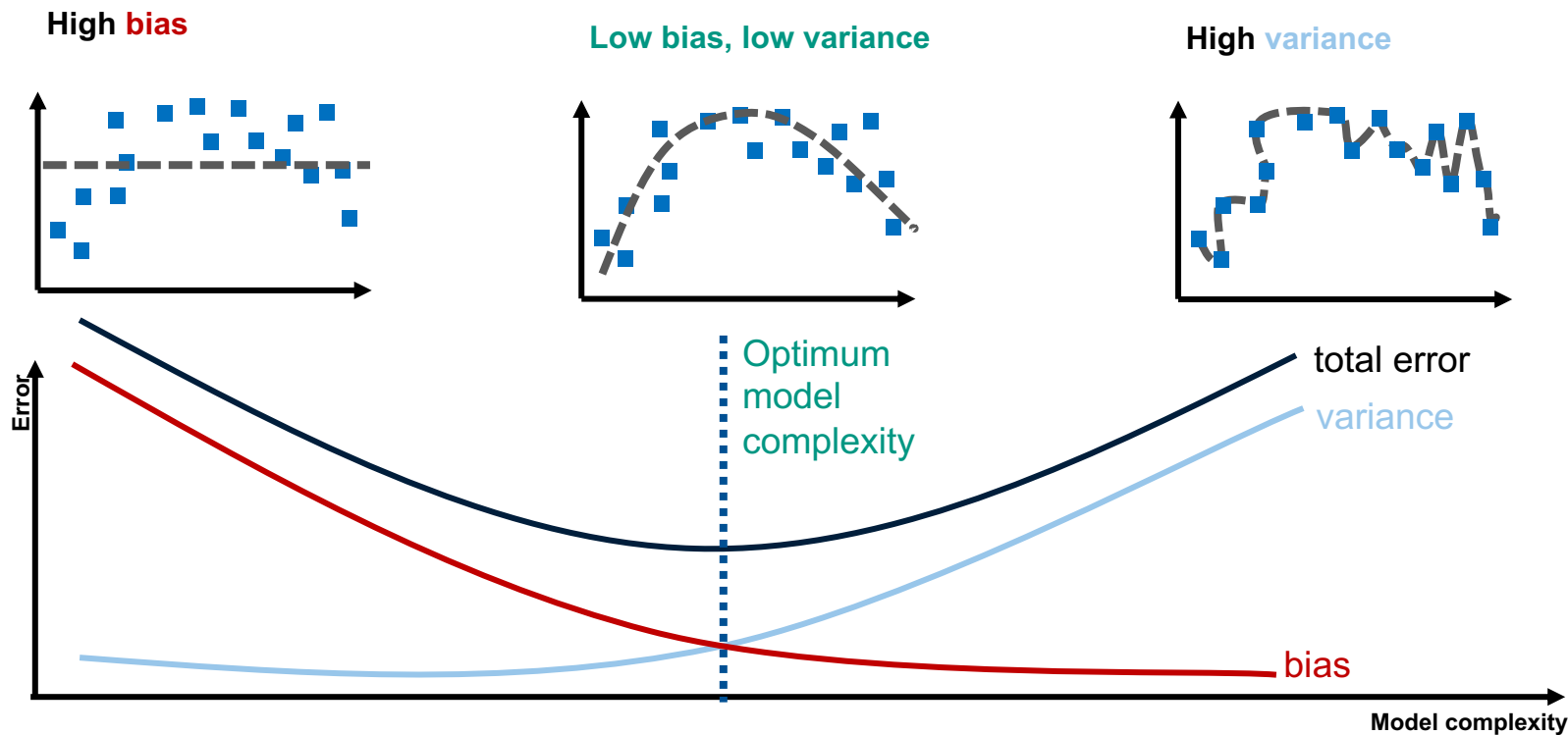
$$\text{Total Damage (US\$)} \approx f(\text{Homeless, Total Death, Injured})$$

We can now decompose the average mean-squared error we expect to make by our estimation of Y as:

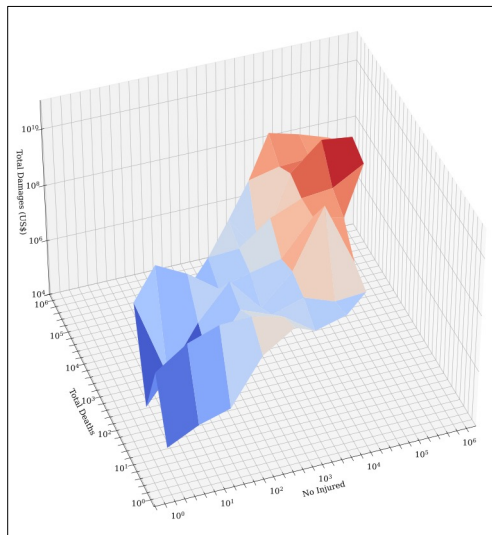
$$\begin{aligned} E(Y - \hat{Y})^2 \\ &= E[f(X) + \varepsilon - \hat{f}(X)]^2 \\ &= \underbrace{[f(X) - \hat{f}(X)]^2}_{\text{Reducible}} + \underbrace{\text{Var}(\varepsilon)}_{\text{Irreducible}} \end{aligned}$$

- Translating business problems to machine learning problems
- Reducible and irreducible error terms in statistical learning
- **Bias variance trade off**
- Classification & Regression
- Data splitting

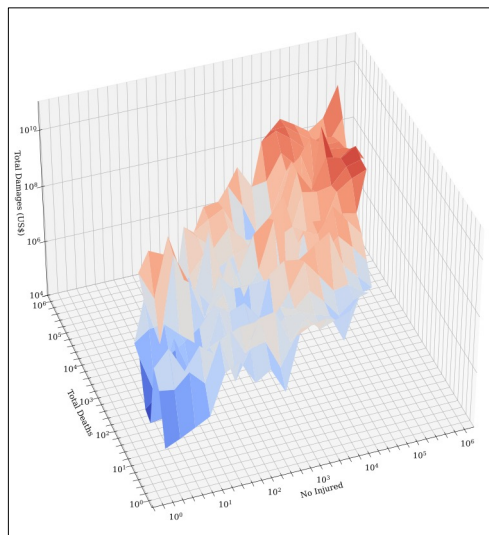
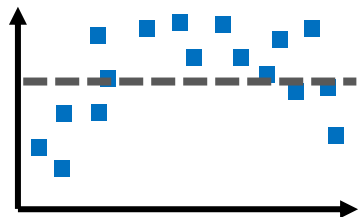
Bias Variance trade-off



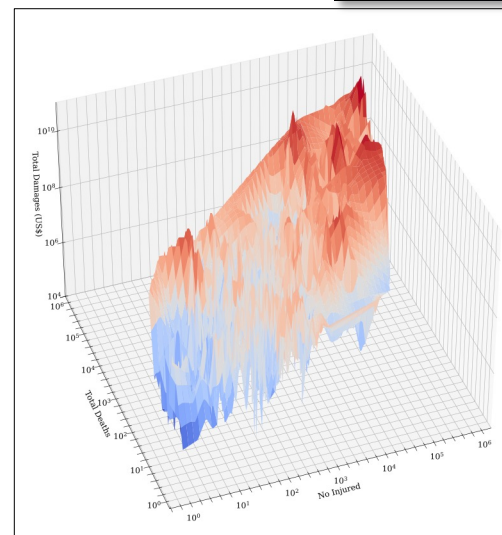
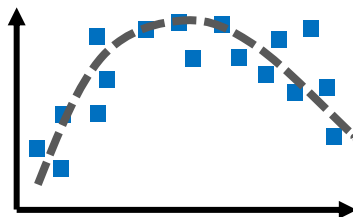
Bias Variance trade-off



High **bias**

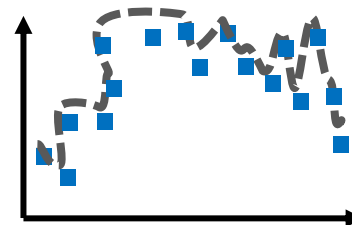


Low bias, low variance



High **variance**

Figure 2.2: function surface



Flexibility of $\hat{f}$

- As the flexibility of $\hat{f}$ increases its variance increases and its bias decreases

Prediction accuracy versus interpretability

- How do we value interpretability compared to higher accuracy?

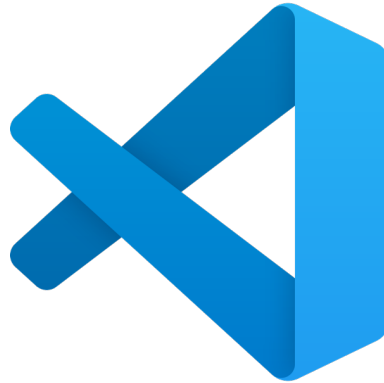
Good fit versus over-fit or under-fit

- How do we know when the fit is just right?

Parsimony versus black-box

- Do we prefer a simpler model involving fewer variables over black-box predictor involving them all?

Bias Variance trade-off.



- Translating business problems to machine learning problems
- Reducible and irreducible error terms in statistical learning
- Bias variance trade off
- **Classification & Regression**
- Data splitting

Measurable Machine Learning Problems

Business Problem



Objectives and business **success criteria** describe the problem. However, these descriptions are often subjective and lack measurability.

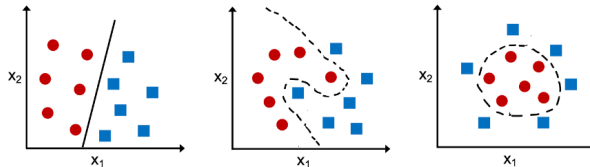
Translation

Machine Learning Problem

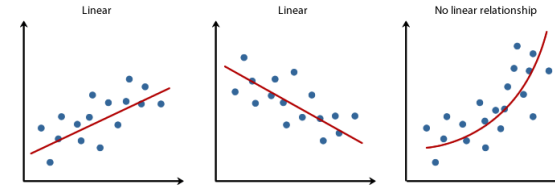


Machine learning algorithms on the other hand require a **clear, measurable goal** that can be quantified by a dominate metric.

Classification | *classify instances into classes*



Regression | *predict numerics as close as possible*



Given a data set $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, we aim to learn a mapping from K-dimensional input vectors (feature vectors) $\mathbf{x}_i$ to outputs $y_i \in \{1, \dots, C\}$, $\forall i \in \{1, \dots, N\}$ with C being the number of classes. If $C = 2$, this is called a binary classification.

We formalize this problem as a function approximation $y = f(\mathbf{x})$.

We learn a statistical model to estimate the function f as an approximation $\hat{f}$ given a labeled training set $\mathbf{x}$. This approximation allows us to make predictions using $\hat{y} = \hat{f}(\mathbf{x})$.

cf. [1]

Note: We can determine the accuracy of the mapping via its error rate, i.e., the proportions of mistakes that the mapping yields

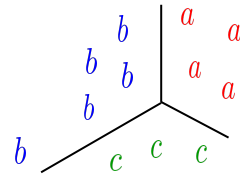
$$\text{error rate} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(y_i \neq \hat{y}_i)$$

Indicator function
equals 1 if $y_i \neq \hat{y}_i$, 0 else

Additional differentiation: Multi-Class vs Binary Classification

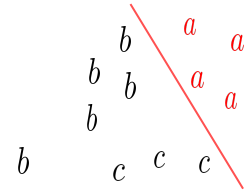
Multi-Class Classification

Classes are mutually exclusive even if the data point is an outlier

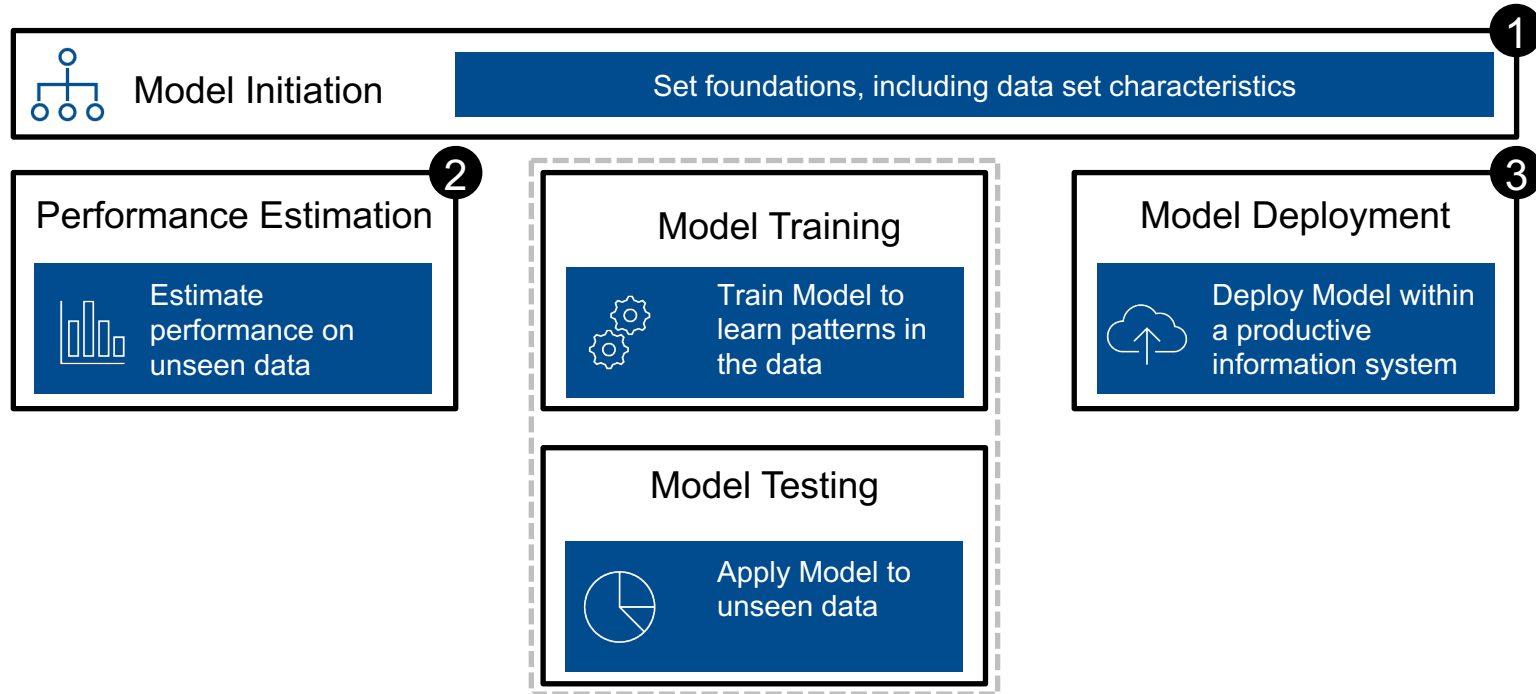


Binary Classification

Distinctions are one-vs-rest, i.e., $\{a\}$ vs $\{\bar{a}\}$



The AI Lifecycle: Required steps for supervised machine learning.



		Prediction	
		Positive	Negative
Ground truth	Positive	True Positives (TP)	False Negatives (FN)
	Negative	False Positives (FP)	True Negatives (TN)

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

Accuracy describes the relation between correct classifications and the total number of input samples and gives a first insight on an ML application's overall performance.

$$\text{Precision} = \frac{TP}{FP+TP}$$

Precision is the proportion of correctly predicted positive cases regarding all positive predicted cases.

$$\text{Recall} = \frac{TP}{FN+TP}$$

Sensitivity/ Recall is the proportion of corrected predicted positive cases regarding all positive cases.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

F1-score is the harmonic mean between precision and recall.

Performance metrics Classification: Selection

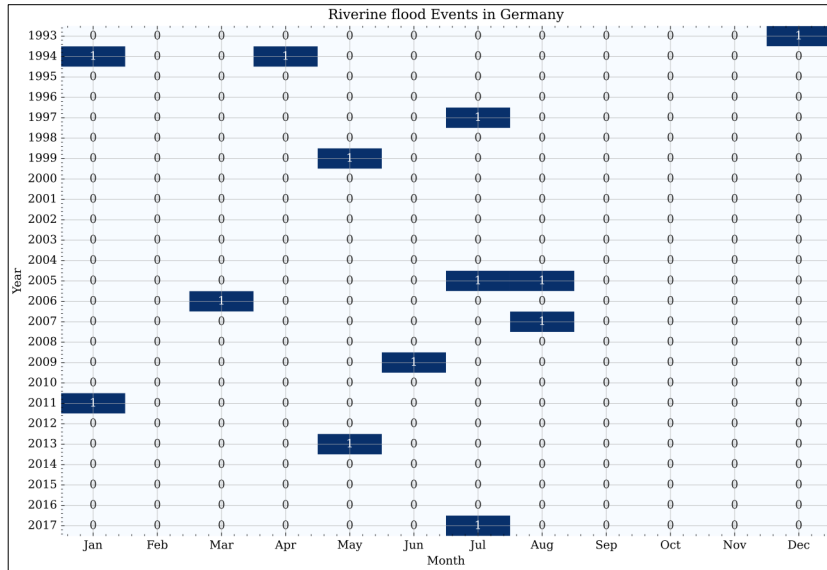


Figure 2.3: joint distribution

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

$$\text{Precision} = \frac{TP}{FP+TP}$$

$$\text{Recall} = \frac{TP}{FN+TP}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$



Which metric
would you
recommend?

Measurable Machine Learning Problems

Business Problem



Objectives and business **success criteria** describe the problem. However, these descriptions are often subjective and lack measurability.

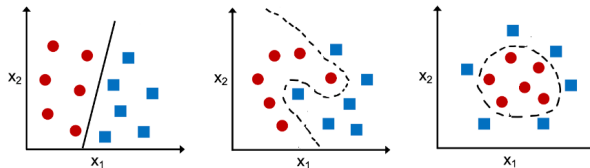
Translation

Machine Learning Problem

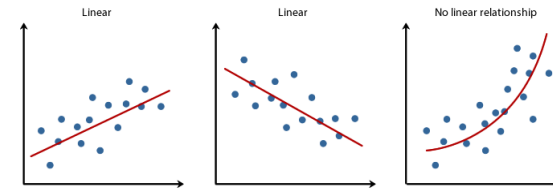


Machine learning algorithms on the other hand require a **clear, measurable goal** that can be quantified by a dominate metric.

Classification | *classify instances into classes*



Regression | *predict numerics as close as possible*



$$\text{MAE} = \frac{1}{N} * \sum_1^N |y_i - \hat{y}_i|$$

Mean absolute error defines the average distance between single data points and the arithmetic mean.

$$\text{MSE} = \frac{1}{N} * \sum_1^N (y_i - \hat{y}_i)^2$$

Mean squared error squares the difference between actual values and predicted values, making MSE helpful when outliers occur as the difference is more penalized.

$$\text{RMSE} = \sqrt{\frac{1}{N} * \sum_1^N (y_i - \hat{y}_i)^2}$$

Root mean squared error is the standard deviation of prediction errors in the model. It states the concentration of data points around the regression line.

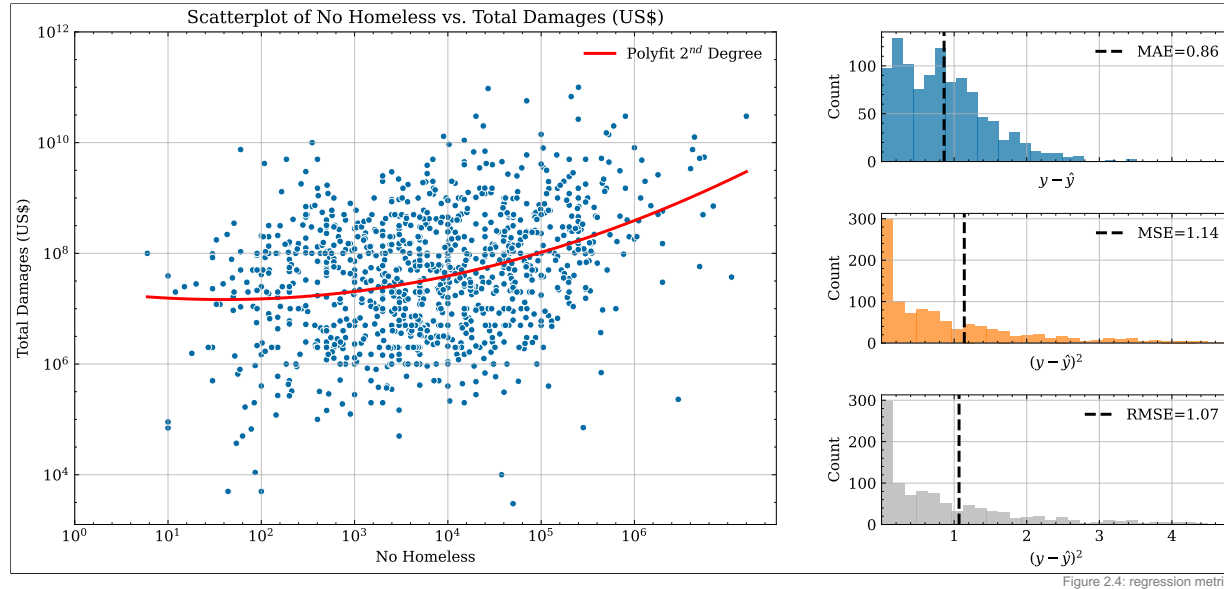
$$R^2 = \frac{\sum_1^N (\hat{y}_i - \bar{y})^2}{\sum_1^N (y_i - \bar{y})^2} = \frac{\text{explained var}}{\text{total var}}$$

R² compares the regression model with a random baseline model and outlines how much variation of a dependent variable is explained by the independent variables.

$\bar{y}$ = arithmetic mean of the estimation, y = (real) single data points, $\hat{y}$ = predicted value for a single data point

Performance metrics Regression: Selection

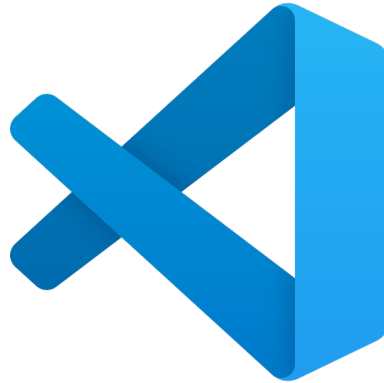
EARTHDATA



$$\text{MAE} = \frac{1}{N} * \sum_1^N |y_i - \hat{y}_i|$$

$$\text{MSE} = \frac{1}{N} * \sum_1^N (y_i - \hat{y}_i)^2$$

$$\text{RMSE} = \sqrt{\frac{1}{N} * \sum_1^N (y_i - \hat{y}_i)^2}$$



- Translating business problems to machine learning problems
- Reducible and irreducible error terms in statistical learning
- Bias variance trade off
- Classification & Regression
- Data splitting

- Training Set:** The data used to train the model, allowing it to learn patterns and relationships.
- Validation Set:** A separate subset of data used during training to tune model parameters and prevent overfitting. It helps improve the model by evaluating performance on unseen data
- Test Set:** A separate subset used to assess the model's performance after training is complete. It provides an unbiased evaluation of the model's ability to generalize to new data.



Often, we see the split of 60% training, and 20% each of validation and testing data – these can vary depending on the specific problem at hand

Training, Validation, and Testing

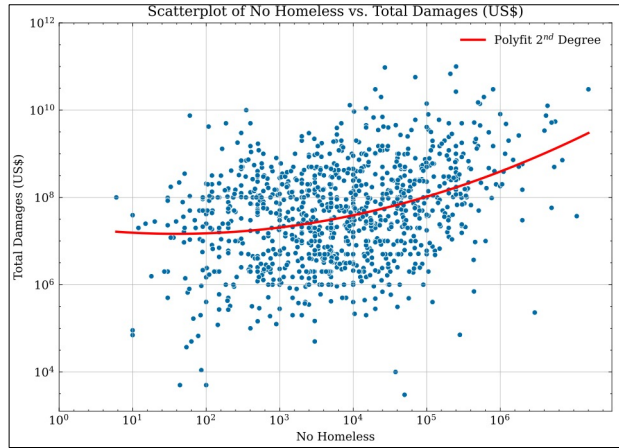


Figure 2.1: feature target relation

?

What split do you suggest?

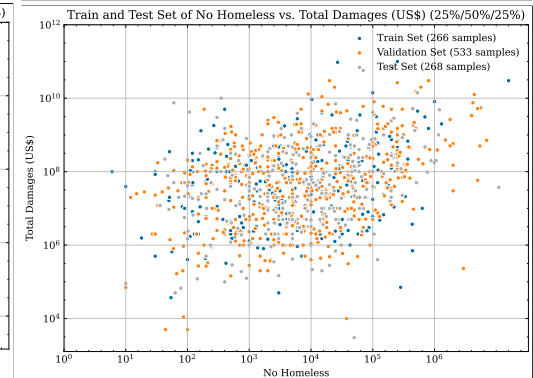
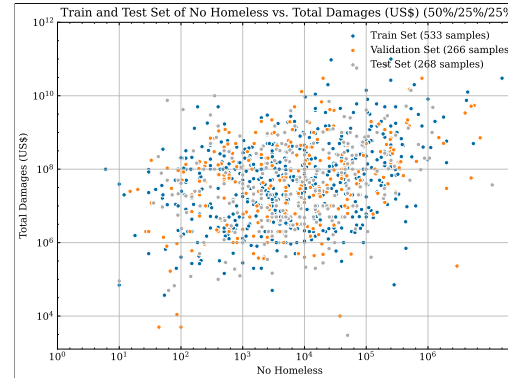
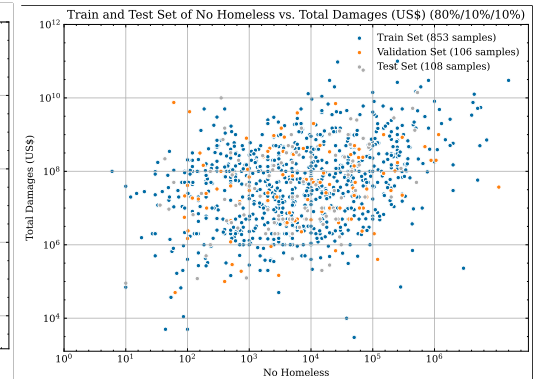
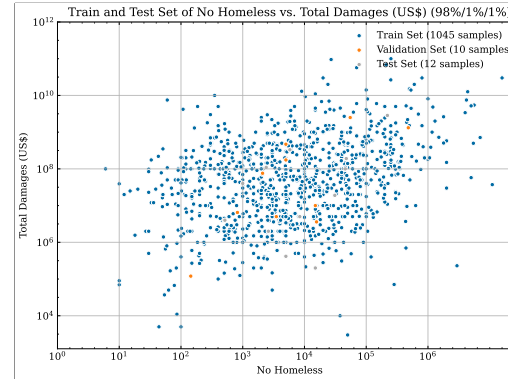


Figure 2.5: train test split

Training Set

Validation Set

Test Set

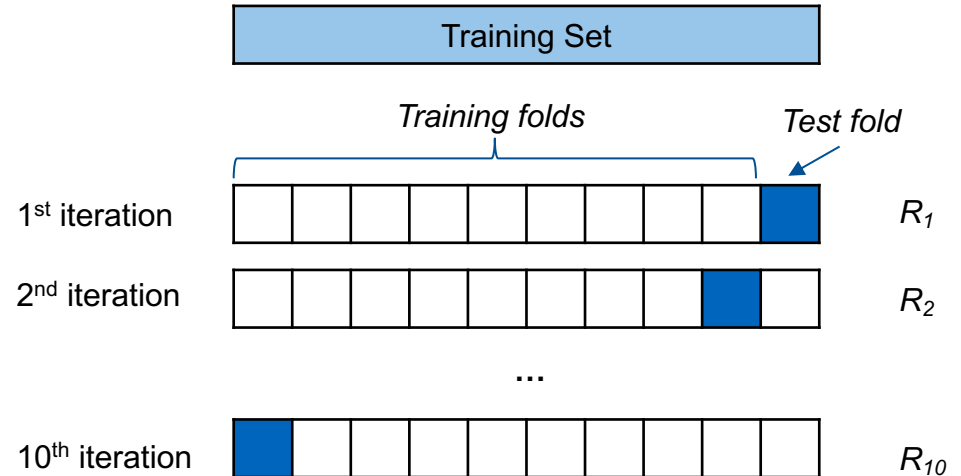
Cross-validation is a technique to evaluate a model's performance by splitting the dataset into multiple training and test sets.

The most common form is **k-fold cross-validation**, where the data is divided into k subsets, and the model is trained and tested k times, each time with a different subset used as test set.

Process:

1. Divide data into k equal subsets.
2. Train the model k times, each time using a different subset as the test set and the remaining $k - 1$ subsets as the training set.
3. Average the results (R_1 to R_k) to get a robust estimate of model performance.

Example: 10-fold Cross-Validation



➔
$$R = \frac{1}{10} \sum_{i=1}^{10} R_i$$

Wrap up.

- Translating business problems to machine learning problems
- Reducible and irreducible error terms in statistical learning
- Bias variance trade off
- Classification & Regression
- Data splitting



What is still **unclear** from this session?